

PortMira

A local-first portfolio mirror · 本地優先的資產鏡像工具

2026 年 5 月

4 人團隊

Python · Streamlit 或 Flask

本地優先 · 隱私保護

PortMira 是一個以 Python 開發的本地優先個人財務儀表板。使用者可以手動輸入資產與負債、透過免費 API 取得即時市場價格、計算淨資產並視覺化資產配置，所有資料皆儲存在本地，不上傳任何資訊至雲端。

動機

個人財務資料往往散落各處——銀行帳戶、券商平台、加密貨幣交易所、試算表，甚至只存在腦海中。要得到一個完整、誠實的財務全貌，通常需要大量手動整理，或者把敏感資料交給隱私政策不透明的商業服務。

市面上的理財工具大多走兩個極端：一種是聚焦消費分類的銀行 App，另一種是以投資預測與主動交易為核心的券商平台。兩者都無法提供一個冷靜、全面的財務現況概覽。PortMira 採取不同的方向——一個簡單的本地儀表板，幫助使用者清楚、私密地看清自己擁有什麼、欠下什麼。

問題定義

目前沒有任何輕量、隱私優先的工具，能在不需要雲端帳號、訂閱方案或不透明資料共享的前提下，將使用者的資產與負債整合為一個可讀的視圖。試算表靈活但需要手動維護公式；理財 App 方便但犧牲隱私；交易儀表板服務的是主動投資者，而不是只想靜靜看清自己淨資產的使用者。PortMira 填補這個空缺。

目標使用者

PortMira 設計給持有多幣種、多類別資產的個人——例如持有台幣或美元現金、美股或台股 ETF，以及加密貨幣——希望在不依賴商業平台的情況下，得到一個清晰的資產總覽。v0.1 的目標使用者是願意自行輸入資料、並在本地執行 Python 應用程式的使用者。

核心產品理念

預設本地優先

隱私保護設計

清晰勝過預測

追蹤而非建議

冷靜而非遊戲化

可運作優先於功能完整

所有使用者資料皆留在本地檔案中。外部 API 僅作為唯讀資料來源，用於取得市場價格。PortMira 不提供任何買進、賣出或持有建議，不顯示預測圖表或交易訊號。介面應給人分析性、值得信任的感受，而非注意力導向或充滿炒作氣氛。AI 功能刻意不納入 v0.1 的範疇，待核心財務資料層穩定後再行考慮。

MVP 功能

資產與負債輸入

- 多幣種現金（台幣、美元、歐元、日圓）
- 股票與 ETF（美股；台股透過 `.TW` / `.TWO` 若可用）
- 加密貨幣持倉
- 選填的手動定價資產（債券、大宗商品等）
- 信用卡負債、個人貸款、保證金借款

儀表板與計算

- 總資產、總負債、淨資產
- 依類別的資產配置圖
- 持倉表：現值、成本、未實現損益、日漲跌
- 幣種切換：台幣 ↔ 美元
- 價格變動警示（使用者自訂門檻：3%、5%、10%）
- 保證金維持率計算機
- 貸款 / 信用卡利息計算機

資產組合資料儲存於本地 JSON 或 CSV 檔案，每次開啟應用程式時自動載入，無需重複輸入。

資料來源與 API

PortMira 僅使用免費公開的 API 作為唯讀資料來源，所有清洗、合併與計算皆由 Python 和 Pandas 處理，應用程式不直接呈現原始 API 回傳結果。

- **股票與 ETF 價格：** `yfinance` 或同等免費套件，涵蓋美股及可用的台股代號。
- **加密貨幣價格：** CoinGecko 公開 API 或其他免費來源（如 CryptoCompare）。
- **外匯匯率：** 免費 FX API（如 ExchangeRate-API、Open Exchange Rates），或在 API 無法使用時切換為靜態備用匯率表。

若任何 API 呼叫因速率限制或網路問題而失敗，應用程式將自動退回至最後一次快取的價格，或使用者手動輸入的價格。即使在沒有網路連線的情況下，應用程式仍應保持可用。

專案將財務邏輯與介面層分離，讓各模組可以獨立測試，未來也更容易遷移至其他框架或平台。

```
portmira/
  app.py                # 主程式入口 (Streamlit 或 Flask)
  src/
    models.py           # 資產與負債資料結構
    storage.py           # JSON / CSV 讀取與儲存
    price_fetcher.py     # yfinance、加密貨幣 API、外匯 API、備用邏輯
    calculations.py      # 淨資產、損益、配置比例、警示系統
    charts.py            # Matplotlib、Plotly 或 Streamlit 圖表
  data/
    portfolio.json       # 本地資產組合資料 (含範例資料)
  docs/
    roadmap.md           # 未來開發計畫
    architecture.md      # 架構說明
  README.md
  requirements.txt
```

UI 使用 **Streamlit 或 Flask** 搭配 HTML/CSS 模板建立，Streamlit 因快速原型開發的優勢為首選。視覺化使用 **Matplotlib、Plotly 或 Streamlit 內建圖表元件**。Pandas 負責所有資料聚合與計算，API 原始回應不直接呈現於介面。

團隊分工

組員	主要負責	主要交付項目
組員 A	資料與儲存層	models.py、storage.py、JSON/CSV 資料格式、範例資料
組員 B	價格取得與 API	price_fetcher.py、API 整合、備用邏輯、快取機制
組員 C	計算邏輯與警示	calculations.py、淨資產、損益、配置比例、警示系統、計算機
組員 D	UI 與儀表板	app.py、charts.py、版面配置、README、展示準備

每位組員負責獨立的模組，介面定義清楚，大幅降低 Git 合併衝突。整合測試安排在第 2 週末與第 4 週末進行。

開發時程

- 第 1 週** 確認資料格式與資產/負債模型、建立 GitHub 儲存庫、確認 API 選擇與模組分工、撰寫範例資產組合資料。
- 第 2 週** 建立儲存層（JSON/CSV 讀寫）、實作價格取得模組（含備用邏輯）、開始核心計算：淨資產、配置比例、未實現損益。
- 第 3 週** 整合里程碑：串接儲存層、價格模組與計算模組；以範例資料開始建立 UI 版面；實作警示系統與基礎計算機。
- 第 4 週** 完成儀表板：持倉表、配置圖、幣種切換、警示顯示、加入匯率換算；以真實情境範例進行內部測試。
- 第 5 週** UI 潤飾、撰寫 README 與文件、準備展示用範例資產組合、排練期末報告。保留緩衝時間修復問題。

風險與應對方案

- **API 失效或速率限制**：應用程式快取最後一次成功的回應，並允許手動輸入價格。核心功能在無網路時仍可正常使用。
- **台股資料缺失**：`yfinance` 對部分台股代號支援可能不完整。手動輸入價格為第一級備用方案，非事後補丁。
- **功能蔓延**：v0.1 嚴格限定在本地儲存、價格取得、儀表板顯示、警示與兩個計算機。任何新功能一律列入 roadmap。
- **視覺化複雜度**：圖表從簡單開始——配置比例用圓餅圖或長條圖，持倉用基本表格。互動功能僅在時間允許時再加入。
- **時程壓力**：若 UI 開發落後，團隊將改以較簡單的介面或 Jupyter Notebook 進行展示，而非延誤可運作的 demo。

未來 Roadmap

以下功能明確**不屬於 v0.1**，但代表專案後續作為 GitHub 側邊專案或產品發展的自然下一步：

- 整理儲存庫、加入截圖、完善 README，以及附有匿名範例資料的公開 demo 模式。
- API 回應快取機制與更智慧的更新頻率控制。
- 支援更多資產類別：債券、大宗商品、REITs、私有資產。
- 從券商匯出的 CSV 檔案匯入（如 Interactive Brokers、富途）。
- 桌面應用程式封裝或行動裝置友善介面。
- 可選的 AI 功能——*僅在核心資料層穩定後才考慮*——例如用白話文總結資產配置、對本地資料進行自然語言查詢（「我有多少美元曝險？」）、情境分析（「如果這檔股票跌 10% 會怎樣？」）、以及月度財務反思提示。這些功能作為解釋層，而非投資顧問。

PortMira 自然而有目的地涵蓋了課程的核心 Python 內容：用字典與串列管理資產組合資料、用函式組織計算模組、用檔案 I/O 實現本地儲存——這些都是課程中反覆練習的基礎概念。API 整合引入了真實世界的資料取得場景，Pandas 實現了乾淨的數值處理與聚合。模組化的架構讓四位組員可以並行開發不同檔案，大幅減少合併衝突。

專案範疇刻意設定為有邊界的：目標是一個可運作、可展示的本地儀表板，而非一個生產級的金融平台。團隊可以在期末報告現場展示真實價格與範例資產組合，呈現方式清楚且具說服力。同時，程式碼架構足夠乾淨，讓專案在課程結束後仍能持續成長為 GitHub 側邊專案——讓這份工作真正有用，而非只是應付作業。